

Smart Contracts

Data: 2026-05-28
Branch: feature/pipeline-guardrails

Mensagem Principal

O projeto implementa um agente hibrido para auditoria e correcao de contratos inteligentes:

```
Slither -> normalizacao -> LLM/planejamento -> CVL -> Certora  
  
-> diagnostico -> correcao -> patch_guard -> revalidacao
```

A LLM nao corrige o contrato "no chute". Ela opera dentro de um pipeline com analise estatica, verificacao formal, correcoes deterministicas e guardrails de patch.

Como O Pipeline Funciona

- 1. Roda Slither no contrato.
- 2. Normaliza achados para classes conhecidas.
- 3. Filtra achados sem propriedade formal confiavel.
- 4. Planeja regras formais e gera spec CVL.
- 5. Certora confirma se a propriedade e violada no contrato original.
- 6. Somente vulnerabilidades confirmadas seguem para diagnostico e patch.
- 7. patch_guard bloqueia patch amplo, truncado ou fora do escopo.
- 8. Certora e Slither revalidam o contrato corrigido.

Contratos Principais

Contrato	Tipo	Tamanho	Run
EnterpriseTreasury300.sol	control ado/	327 linhas	runs/ 20260527_095632_Ente

Contrato	Tipo	Tamanho	Run
	stress		Treasury300
Forgotten Runes	real Code4 rena	631 linhas	runs/ 20260527_105459_Forg unesWarriorsMinter
Inverse Market.sol	real Code4 rena	621 linhas	runs/ 20260528_093346_Mark
Caviar PrivatePool.sol	real Code4 rena	794 linhas	runs/ 20260528_101335_Priv l
Solmate WETH.sol	real/ control e negativ o	pequ eno	runs/20260527_102842
Escher FixedPrice.sol	real/ triage m selfdes truct	112 linhas	runs/ 20260528_093456_Fixe

Melhor Caso Para Mostrar: Inverse Market

Arquivo: smart-audit/contracts/real/inverse/src/Market.sol

Problema encontrado: tres funcoes criticas aceitavam address(0):

- setGov(address)
- setLender(address)
- setPauseGuardian(address)

O Slither identificou os achados, o Certora confirmou as violacoes e o pipeline aplicou patch minimo.

```
function setGov(address _gov) public onlyGov {

    require(_gov != address(0), "Zero address"); // FIX VULN_023

    gov = _gov;

}

function setLender(address _lender) public onlyGov {

    require(_lender != address(0), "Zero address"); // FIX VULN_026

    lender = _lender;

}

function setPauseGuardian(address _pauseGuardian) public onlyGov {

    require(_pauseGuardian != address(0), "Zero address"); // FIX VULN_020

    pauseGuardian = _pauseGuardian;

}
```

Resultado: **3/3 resolvidos**. Patch guard: 3 hunks, 0 erros, 0 avisos.

Contrato De 794 Linhas: Caviar PrivatePool

Arquivo: smart-audit/contracts/real/caviar/src/PrivatePool.sol

Esse contrato compilou com Slither usando imports reais e remappings.txt de Foundry, mas nao tinha achado dentro das classes que o pipeline trata com confianca.

Resultado: 0 candidatos acionaveis. Isso e um controle negativo: o agente nao inventou vulnerabilidade nem gastou Groq.

Comandos Para Demo Sem Gastar Groq

```
cat runs/20260528_093346_Market/comparison_t1.json

cat runs/20260528_093346_Market/patch_guard_t0.json

sed -n '126,152p' agent/agent_outputs/Market_FIXED.sol
```

Rodar ao vivo um caso seguro, que deve parar antes da LLM:

```
python3 agent/main.py --contract
smart-audit/contracts/real/caviar/src/PrivatePool.sol
```

Opcao Com Correcao Ao Vivo

```
python3 agent/main.py --contract
smart-audit/contracts/real/inverse/src/Market.sol
```

Aviso: esse caso pode chamar Groq na etapa de planejamento formal. Para a apresentacao, e mais seguro mostrar o run ja salvo.

Limites Atuais

- Reentrancia ainda nao esta no automatico.
- Vulnerabilidades economicas e oraculos exigem invariantes de negocio.
- tx.origin precisa distinguir autorizacao vulneravel de politica EOA/allowlist.

- Repos reais podem bloquear por dependencias externas indisponiveis.

Frases Para Apresentar

- O objetivo nao e substituir auditoria humana, mas reduzir trabalho repetitivo e criar um ciclo verificavel.
- O pipeline so corrige depois que uma propriedade e confirmada.
- Casos negativos mostram que o agente nao esta apenas inventando vulnerabilidades.
- Padroes mecanicos foram movidos para regras deterministicas para economizar tokens e reduzir variancia.